R1.01

CM-02

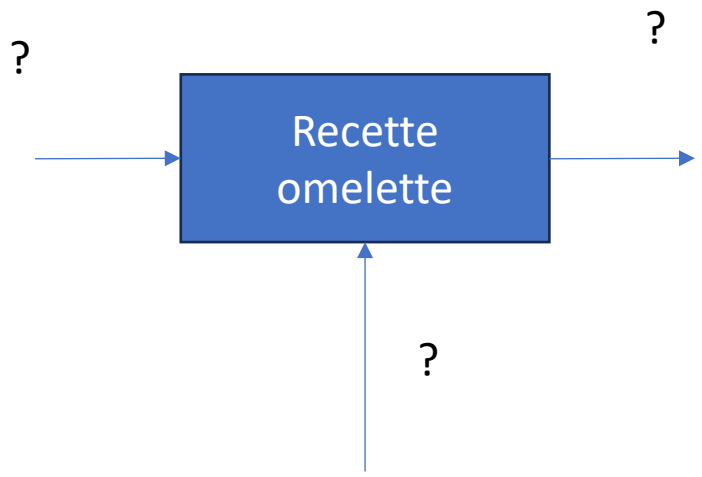
Algorithmique - bases

Basé sur le cours de Loïg Jezequel

Définition

Un algorithme est une méthode générale pour résoudre un type de problèmes. Il est dit correct lorsque, pour chaque instance du problème, il se termine en produisant la bonne sortie, c'est-à-dire qu'il résout le problème posé.

Algorithme

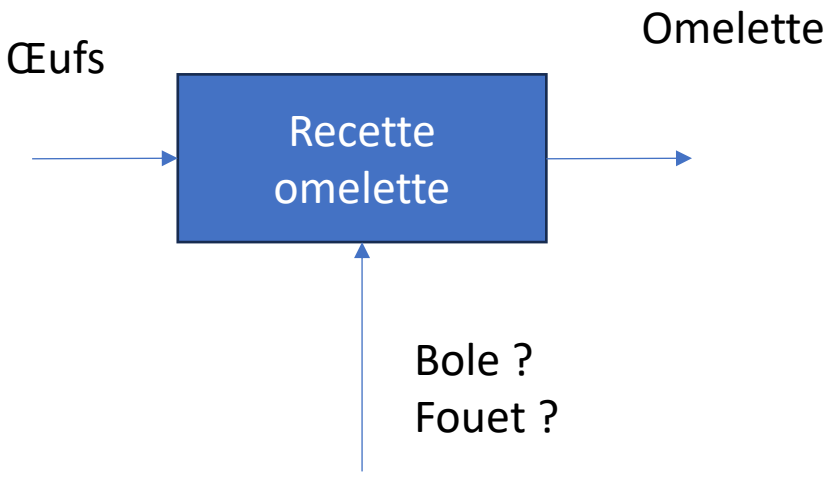


Une recette de cuisine : une omelette

Entrée : ?

Sortie : ?

Algorithme



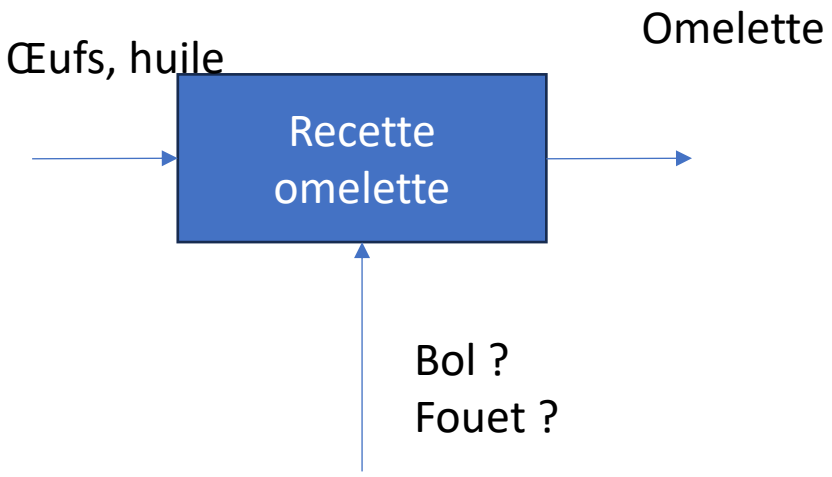
Une recette de cuisine : une omelette

Entrées : Œufs

Sortie : une omelette

Des opérations : ?

Algorithme



Une recette de cuisine : une omelette

Entrées : Œufs
Sortie : une omelette

Des opérations **séquence**

1. Casser les œufs dans un bol
2. Battre les œufs
3. Mettre les œufs avec de l’huile dans la poêle
4. Faire cuire

Algorithme

1. Casser les œufs dans un bol
2. Battre les œufs
3. Mettre les œufs avec de l’huile dans la poêle
4. Faire cuire

Conditionnelle

1. Casser les œufs dans un bol
2. Battre les œufs
3. **Si** il y a de l’huile, mettre les œufs avec de l’huile dans la poêle **sinon** avec du beurre
4. Faire cuire

Algorithme

1. Casser les œufs dans un bol
2. Battre les œufs
3. **Si** il y a de l’huile, mettre les œufs avec de l’huile dans la poêle **sinon** avec du beurre
4. Faire cuire

Boucle

1. Casser les œufs dans un bol
2. Battre les œufs
3. Si il y a de l’huile, mettre les œufs avec de l’huile dans la poêle sinon avec du beurre
4. Faire cuire **tant que** les œufs ne sont pas cuits

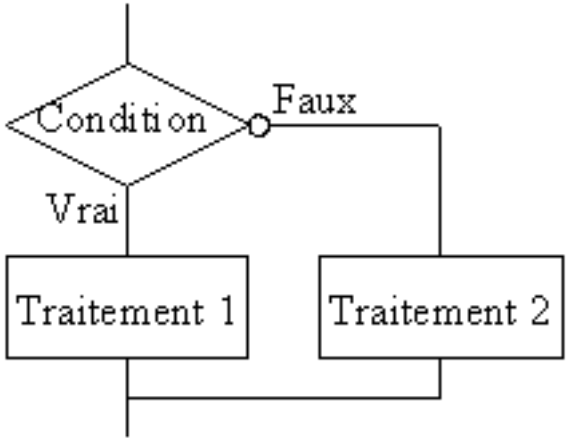
Algorithme

Identifier les entrées et les sorties,

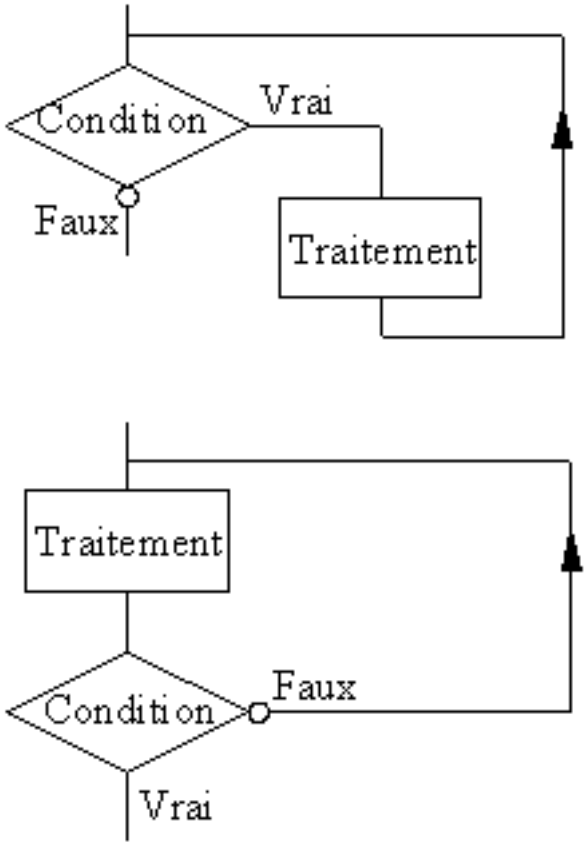
Raffiner en commençant par la séquence, puis les boules et les conditionnelles extérieur, répéter le processus sur les parties imbriquées

Algorithmique : logigramme

Séquence



Conditionnelle



Boucles

Algorithmique : pseudo langage

- La séquence comme une suite de ligne
- Notion de bloc début / fin
- Si (cond) Alors instruction Sinon instruction finsi
- Tantque (condition) faire instruction fin tantque

…

Algorithmique : spécification formelle

Repose sur un formalisme mathématique : Exemple méthode B

Notions de spécification, de preuve, de raffinement

```
MACHINE
        swap
VARIABLES
        xx, yy
INVARIANT
        xx : NAT & yy : NAT
INITIALISATION
        xx :: NAT ||
        yy :: NAT
OPERATIONS
        echange =
        BEGIN
                xx := yy
        ||      yy := xx
        END
END
```

```
REFINEMENT
        swapR
REFINES
        swap
VARIABLES
        xr, yr, zr
INVARIANT
        xr= xx & yr = yy & zr : NAT
INITIALISATION
        xr, yr, zr:= 0, 0, 0
OPERATIONS
        echange =
        BEGIN
                zr := xr
        ;       xr := yr
        ;       yr := zr
        END
END
```

Algorithme : Faisons une addition

Problème : ?

Entrées : ?

Sortie : ?

Principe de l’algorithme : ?

Une instance : ?

Algorithme : Faisons une addition

Problème : Etant donnés x et y deux nombres entiers positifs exprimés en base 10, calculer la valeur de la somme x + y.

Entrées : x et y

Sortie : y

Principe de l’algorithme : ?

Une instance (une exécution possible): ?

Algorithme : Faisons une addition

Problème : Etant donnés x et y deux nombres entiers positifs exprimés en base 10, calculer la valeur de la somme x + y.

Entrées : x et y

Sortie : y

Principe de l'algorithme : Additionner les chiffres deux `a deux, de droite à gauche, en propageant les éventuelles retenues.

Une instance (une exécution possible): x=234, y =56

```
  2 3 4      2 ¹3  4      2 ¹3  4    2 ¹3   4
+   5 6    +   5  6    +   5  6   +   5  6
---------  -----------  -----------  -----------
                   0         9 0    2  9  0
```

Algorithme : Faisons une addition

Notation

$x = x_n x_{n-1} \ldots x_0$, $y = y_m y_{m-1} \ldots y_0$ avec sans perte de généralité $m <= n$

Algorithme :

variables : $i=0$, $r_0=0$

Tant que $i \leq m$ poser $z_i = (x_i + y_i + r_i)_0$ et $r_{i+1} = (x_i + y_i + r_i)_1$ puis augmenter i de 1.

Tant que $i \leq n$ poser $z_i = (x_i + r_i)_0$ et $r_{i+1} = (x_i + r_i)_1$ puis augmenter i de 1.

Si $r_{n+1} = 0$, retourner le nombre $z_n z_{n-1} \ldots z_1 z_0$ et sinon (si $r_{n+1} > 0$), retourner le nombre $r_{n+1} z_n z_{n-1} \ldots z_1 z_0$.

$$
\begin{array}{lcccc}
 & x_n & x_{n-1} \ldots & & x_0 \\
+ & r_{n+1} & r_n \; r_{n-1} \ldots & & r_0 \\
 & & y_m \; y_{m-1} \ldots & & y_0 \\
\hline
 & x_n & x_{n-} & \ldots & z_0
\end{array}
$$

Algorithmique : calculons un pgcd

Plus Grand Commun Diviseur de deux entiers

Entrée : a,b deux entiers

Sortie : le pgcd

Algorithme :

Tant que b != 0, poser b= a mod b et a = b. Retourner a.

Une instance du problème : a = 594 et b = 459

b = 594 % 459 = 135, a = 459

b = 459 % 135 = 54, a = 135

b = 135 % 54 = 27, a = 54

b = 54 % 27 = 0, a = 27

b = 0, on retourne 27

Rem :

594 = 22 * 27

459 = 17 * 27

Algorithme exemple

Biologie

Beaucoup de problèmes autour de l'ADN et notamment le problème d'alignement de s´séquences ADN.

Internet

Problèmes de routage, de recherche et classement de données, de compression.

Industrie

Problèmes de logistique et planification avec optimisation des ressources.

Remarque

Les algorithmes pour résoudre ces problèmes sont trop complexes pour les traiter dans ce cours, mais les techniques qu'on abordera sont à la base de ces algorithmes.

Pourquoi étudier l'algorithmique ?

Evolution des performances des ordinateurs

Tous les deux ans environ le nombre de transistors dans les micro-processeurs double (loi de Moore) et donc la puissance de calcul, la mémoire augmente aussi rapidement.

Mais. . .

exécuter un algorithme a quand même toujours un coût en énergie et en temps,

et certains problèmes sont intrinsèquement trop complexes pour ˆêtre résolus par force brute.

Donc savoir concevoir des algorithmes performants est important.

Algorithmes performants : un exemple

Cormen et. al, les tris seront abordés lors d'un prochain cours

Cas A

Rangement de n éléments par tri par insertion bien programmé :

$2n^2$ opérations, sur un ordinateur puissant : 1 milliard d'opérations par seconde.

Cas B

Rangement de n éléments par tri fusion mal programmé :

$50n \log_2(n)$ opérations, sur un ordinateur peu puissant : 10 millions d'opérations par seconde.

Si n = 106

Dans le cas A l'exécution n´nécessite $2.(10^6)^2$ opérations et dure donc $2.(10^6)^2/109$ = 2000 secondes alors que dans le cas B l'exécution nécessite $50.10^6. \log_2(10^6)$ opérations et dure donc $50.10^6. \log_2(10^6)/107$ = 100 secondes.

Pourquoi ´etudier l’algorithmique ? suite et fin

Même avec une puissance de calcul et une mémoire infinies

- Terminaison : Un algorithme mal conçu peut boucler sans arrêt, et

donc ne jamais donner de réponses.

- Validité : Un algorithme mal conçu peut donner des réponses incorrectes.
- Implantation : Un algorithme mal compris peut être mal codé, le code exécuté par l’ordinateur ne correspondra alors pas à ce qui est attendu (terminaison et validité ne sont plus assurées).

Pour aller plus loin

L'efficacité d'un algorithme est mesurée notamment par :

- sa durée de calcul (en partant du principe que chaque instruction a un temps d'exécution constant) ;
- sa consommation de mémoire vive ;
- la précision des résultats obtenus (par exemple avec l'utilisation de méthodes probabilistes) ;
- sa scalabilité (extensibilité);
- sa parallélisation ;
- son ergonomie et en particulier sa contrôlabilité son introspectabilité (introspection) ;
- sa robustesse, résilience ou antifragilité au bruit, aux chocs et en particulier l'émergence et le chaos ;